CS 432: Databases

# Assignment 03
# Module B

*Track 01*

*by*

Nitin Bansal
Devvrat Hans
Sailaja Mudu
Thipparapu Rushitha
Vadithya Harsha Vardhan Nayak
**of Group 21**

Bachelor of Technology,
Indian Institute of Technology Gandhinagar

**GitHub repo Link:-**
**https://github.com/devvrat-hans/CS-432-2026/tree/main/assignment-03/Module_B**

**Video link:-**
**https://drive.google.com/file/d/10qiafy3X7TkJf0Waqy5wL7VZtjm28j95/view?usp=sharing**


## Abstract

This report presents the implementation and validation of Module B for Assignment 03, focused on correctness under concurrent usage, race-condition safety, failure rollback, and stress reliability. The system is built as a transaction-aware API over SQLite with role-based access control, audit logging, and resilience endpoints for controlled failure injection. The validation strategy uses automated multi-user tests that simulate simultaneous access, conflict-heavy operations, rollback scenarios, and high-load workloads. Results show correct atomic behavior, preserved consistency, safe isolation between users, and durable persistence of committed transactions. No invariant violations were observed in the recorded runs, including thousands-scale insert tests with measurable latency and full correctness under contention.


## Introduction

Module B evaluates whether the system remains correct and reliable when many users interact with shared data concurrently. The core objective is not only feature completion, but correctness under realistic conflict and failure conditions.
This module verifies the following required properties:

- Concurrent usage correctness under parallel operations.
- Race-condition safety for critical shared operations.
- Failure rollback with no partial data persistence.
- Stress reliability for hundreds to thousands of requests.
- ACID-aligned behavior: Atomicity, Consistency, Isolation, Durability.

The implementation uses a backend API with transactional operations and a structured test suite that reproduces concurrency conflicts, injected failures, and load spikes in a deterministic and repeatable way.

## System Overview

The Module B system provides:

- Authenticated multi-user API access with admin and user roles.
- Session-based access control and RBAC enforcement.
- CRUD operations over project databases, tables, and records.
- Resilience endpoints for one-time-token workflows and rollback testing.
- Audit logging to both persistent database logs and file logs.
- Index-aware queries and record-key ordering support for fast retrieval and consistency checks.

Correctness controls implemented in the backend:-

**Database constraints:**

- Unique keys on critical identifiers.
- Foreign key constraints with enabled FK enforcement.
- Check constraints for valid status and domain values.

**Transaction control:**

- Explicit transaction boundaries in critical consumer operations.
- Rollback on failure and conflict detection.
- Commit only after all dependent updates succeed.

**Concurrency safety:**

- Immediate transaction locking for critical token consumption paths.
- Conditional update semantics to enforce single-winner outcomes.
- Observability:
- Structured audit events for success, failure, and denied actions.

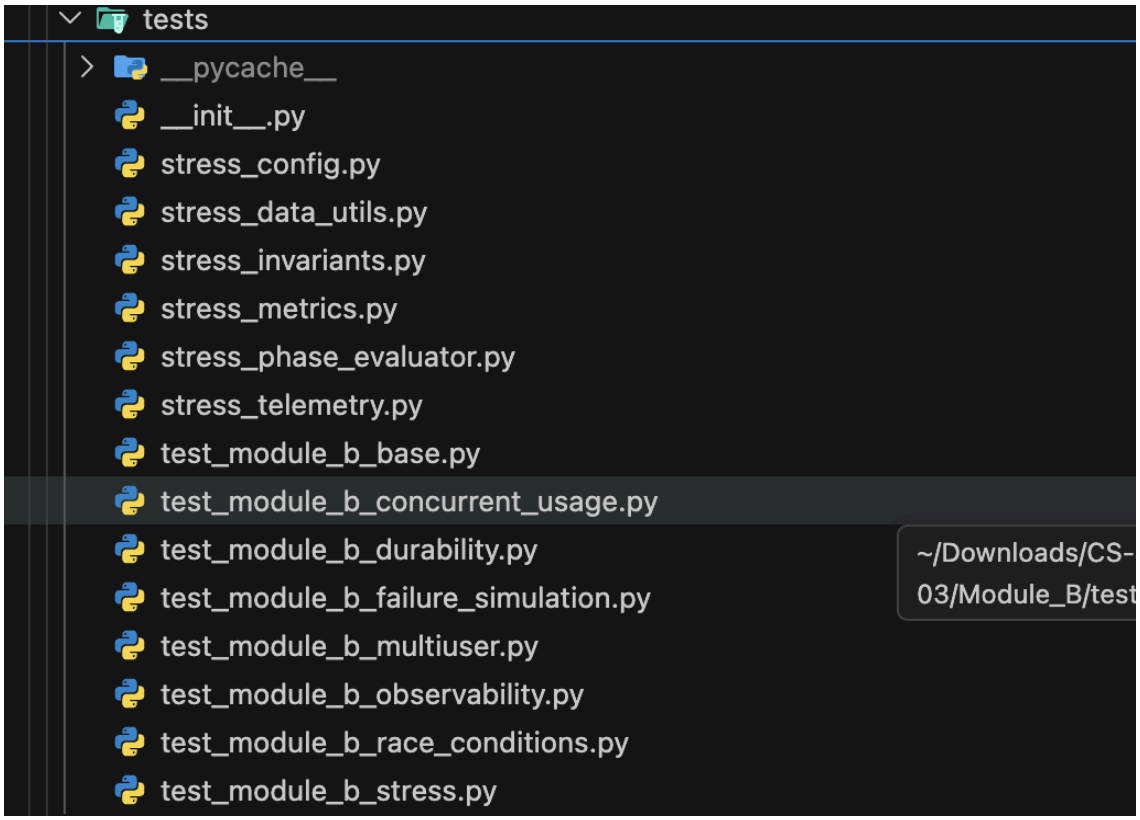
## Assignment Setup

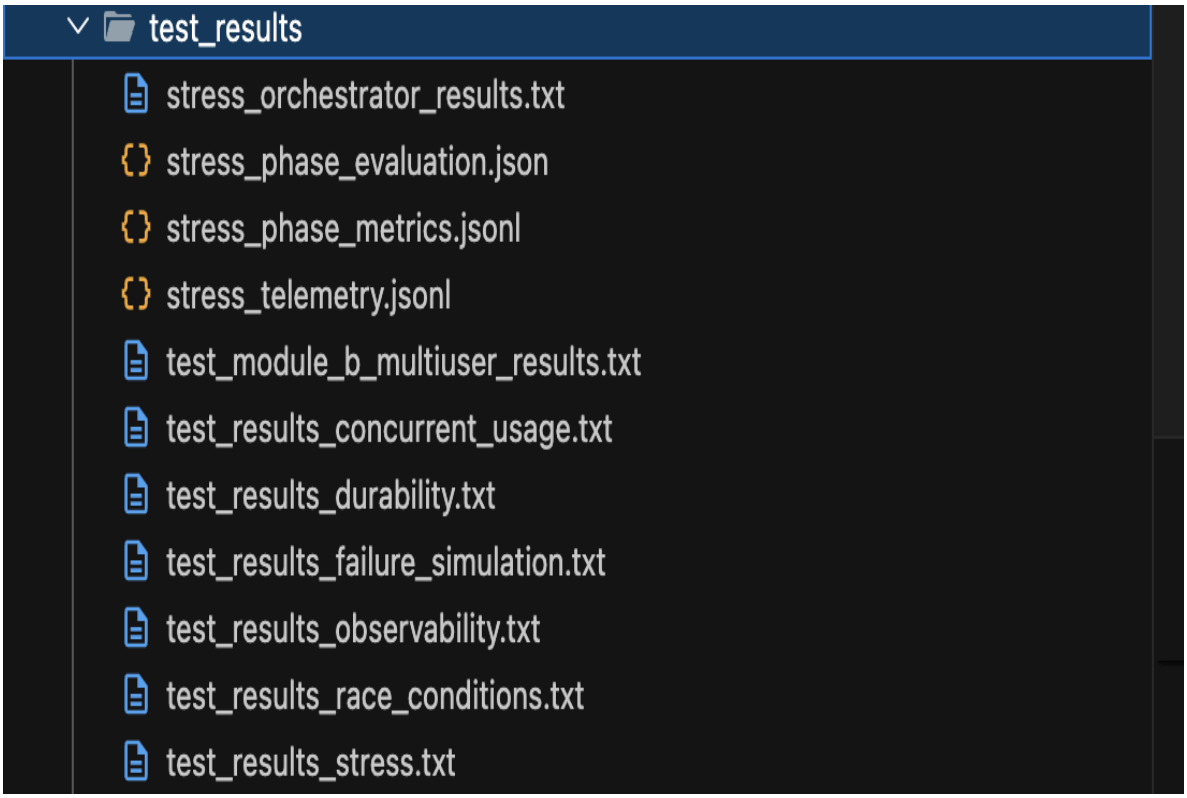
**Language and runtime:** Python 3.13.
**Backend storage:** SQLite.
**API framework:** Flask.
**Test approach:** automated scripts using parallel worker threads.
**Execution mode:** local backend test client with concurrent request simulation.

**Method:**

- Run dedicated test suites for each objective: concurrency, race, failure, durability, observability, stress, and multi-user behavior.
- Record pass/fail and elapsed time per suite.

- Validate both API responses and persistent database state.
- Enforce invariants such as unique records, no orphan references, and exact final counts.



## Results

**Concurrent Usage Testing:- verify that simultaneous users do not corrupt shared state.**

**Workloads validated:**

1. **Parallel unique inserts:**
    - 300 insert requests with 20 workers.
    - Expected: all 300 succeed, final count exactly 300.
2. **Reads during writes:**
    - 150 writes plus 40 read samples in parallel.
    - Expected: successful reads and final count exactly 150.
3. **Duplicate database creation race:**
    - 18 concurrent create attempts for the same name.
    - Expected: 1 success, remaining rejected safely.
4. **Mixed create-update-delete contention:**
    - 180 operations across 24 workers on the same record key.
    - Expected: legal outcomes only, no duplicates, final count in valid set.
5. **Cross-table contention:**
    - Parallel token consumption and token fixture creation.
    - Expected: no referential integrity violations and valid token states

**Recorded suite result:**

Total: 5

PASS: 5

FAIL: 0

ERROR: 0

Elapsed seconds: 2.0043

**Race Condition Testing:- ensure critical shared operations produce exactly one valid winner.**

**Critical races tested:**

- Single token consumption under 40 parallel attempts.
- Expected: 1 success, 39 conflict responses.

- Duplicate record-key insert under 30 parallel attempts.
- Expected: 1 success, 29 duplicate rejections.
- Repeated randomized race rounds.
- 10 rounds, varying worker count.
- Expected: deterministic invariants hold every round.

**Recorded suite result:**
Total: 3
PASS: 3
FAIL: 0
ERROR: 0
Elapsed seconds: 0.8169

**Interpretation:**
The system prevents double-consumption and duplicate-key corruption under high contention, confirming single-winner semantics.

**Failure Simulation and Rollback:-verify atomic rollback and no partial persistence when faults occur.**

**Failure-injection stages tested:**

- before_status_update
- after_token_update
- before_commit

**Validation checks:**

- Token remains ACTIVE after failed attempt.
- Download count remains 0 after failure.
- Upload session status remains ACTIVE after failure.
- Retry after failure succeeds exactly once.
- Second retry returns conflict and does not duplicate effects.

**Recorded suite result:**

Total: 5
PASS: 5

FAIL: 0
ERROR: 0
Elapsed seconds: 0.0506

**Interpretation:**
All tested failure points rolled back cleanly, proving no partial commits.

**Durability Testing:-prove committed state persists across restart simulation and failed state does not leak.**

**Method:**

- Perform commit path on token consumption.
- Validate state from a fresh Python process reading the same database.
- Perform failed transaction injections.
- Revalidate from a fresh process that state remains pre-failure.

**Recorded suite result:**
Total: 2
PASS: 2
FAIL: 0
ERROR: 0
Elapsed seconds: 0.1032

**Interpretation:**
Committed data survives process restart simulation, and failed transactions remain absent after restart checks.

**Observability and Audit Integrity:-ensure operational visibility for success, failure, and access denial.**

**Validated audit paths:**

- Successful mutation logging.
- Failure logging with exception details.
- RBAC denied operation logging.

- Required audit fields present and parseable timestamps.

**Recorded suite result:**

Total: 1
PASS: 1
FAIL: 0
ERROR: 0
Elapsed seconds: 0.0120

**Interpretation:**
Audit records are complete enough for debugging, compliance tracking, and test traceability.

**Stress Testing:-validate correctness and response behavior under sustained heavy load.**

**Stress workloads:**

**High-volume parallel inserts:**
- 600 requests, 30 workers.

**Many token consumptions under load:**
- 120 tokens, 35 workers.

**Thousands-scale insert metrics:**
- 1000 requests, 24 workers.

**Captured metrics:**
- success_rate = 1.0000
- invariant_violations = 0
- average_latency_ms = 50.778
- p95_latency_ms = 227.092
- total_seconds = 2.311

**Sustained read stress:**
- 700 repeated reads over seeded records.
- Expected stable counts and bounded average latency.

**Recorded suite result:**

Total: 7
PASS: 7
FAIL: 0
ERROR: 0
Elapsed seconds: 5.5362

**Interpretation:**
The system maintained correctness and strong response characteristics across high request volumes.

**Multi-User Behavior Validation:-verify role isolation and conflict safety in mixed user populations.**

**Scenarios:**

- Race single-consumer winner validation.
- Failure rollback with no partial residue.
- Hundreds-scale insert stress from parallel workers.
- Parallel admin and regular-user sessions:
- Admin update operations succeed.
- Regular-user unauthorized deletes are denied.
- The shared record remains valid.

**Recorded suite result:**

Total: 4
PASS: 4
FAIL: 0
ERROR: 0
Elapsed seconds: 1.4313

**Interpretation:**
Users do not interfere beyond allowed permissions, and RBAC isolation remains stable during concurrency.

## Consolidated Results Summary

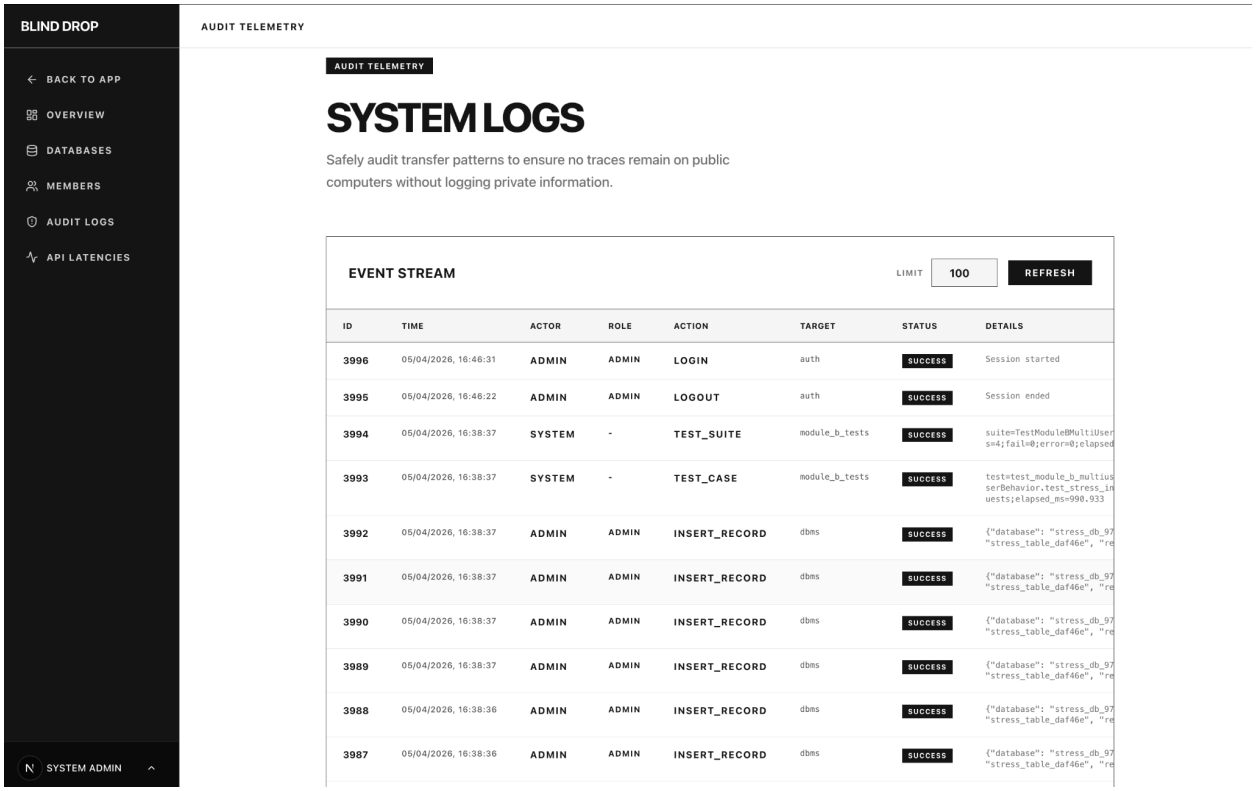
| Test Suite | Total tests | Passed | failed | errors | Execution time(s) |
|---|---|---|---|---|---|
| Concurrent Usage | 5 | 5 | 0 | 0 | 2.0043 |
| Race Conditions | 3 | 3 | 0 | 0 | 0.8169 |
| Failure Simulation | 5 | 5 | 0 | 0 | 0.0506 |
| Durability | 2 | 2 | 0 | 0 | 0.1032 |
| Observability | 1 | 1 | 0 | 0 | 0.0120 |
| Stress | 7 | 7 | 0 | 0 | 5.5362 |
| Multi-User | 4 | 4 | 0 | 0 | 1.4313 |
| **Overall** | **27** | **27** | **0** | **0** | **9.9545** |

**Testing Audit Logs :-**

**Note: -** To observe the audit logs generated during testing, the backend database configuration was modified to **test_results/module_b_test_runtime.sqlite3** database. This is so because testing was purposefully run on an isolated sqlite file at **test_results/module_b_test_runtime.sqlite3**, so stress tests do not pollute the main runtime catalog and also ensure that the stress and automated tests do not interfere with the main runtime database located at: **db_management_system/module_b.sqlite3.**

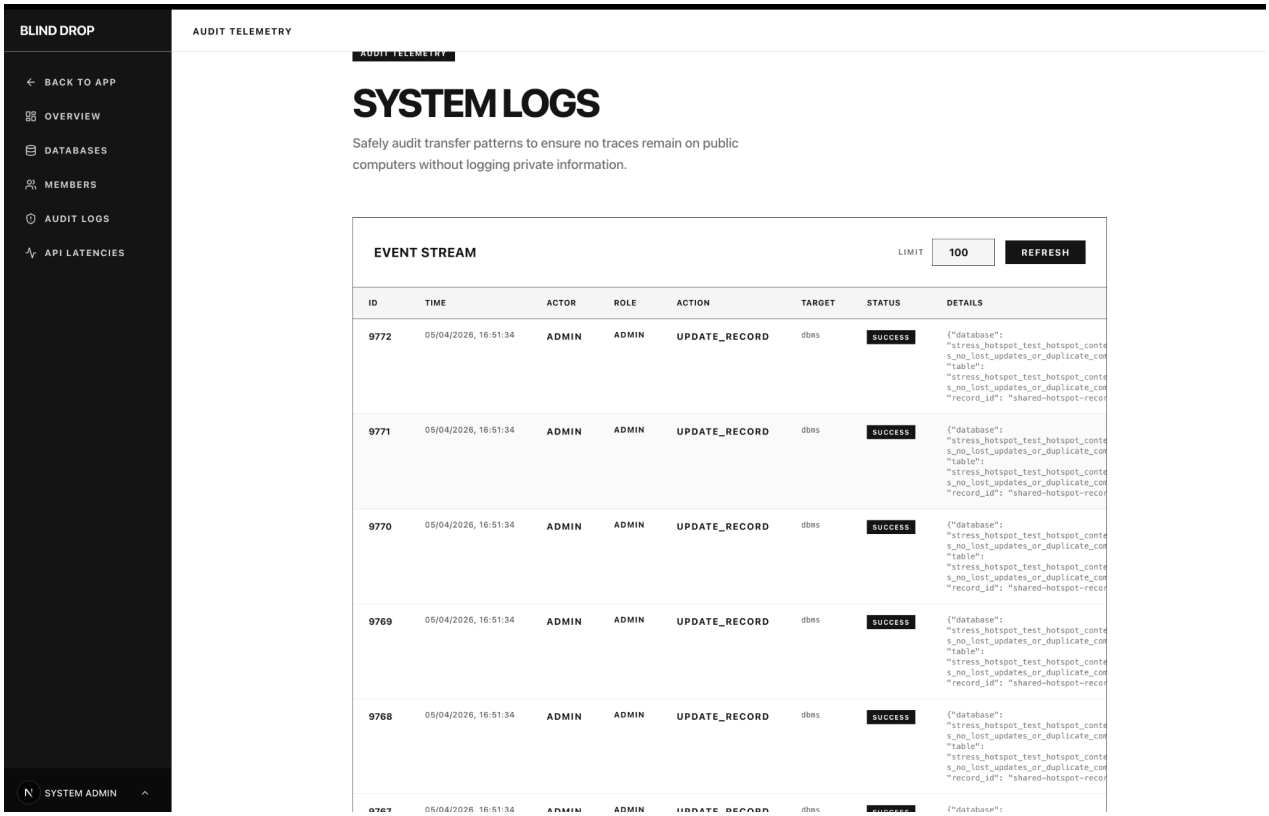
1. All tests executed through the Python script **run_module_b_tests.py** are recorded in the audit logs. These logs confirm that each testing script is successfully interacting with and validating the BlindDrop backend and frontend components.

As shown in the image, running the test script generated a total of 3996 audit log entries, demonstrating that the testing framework actively records and verifies each operation performed during the automated test execution.



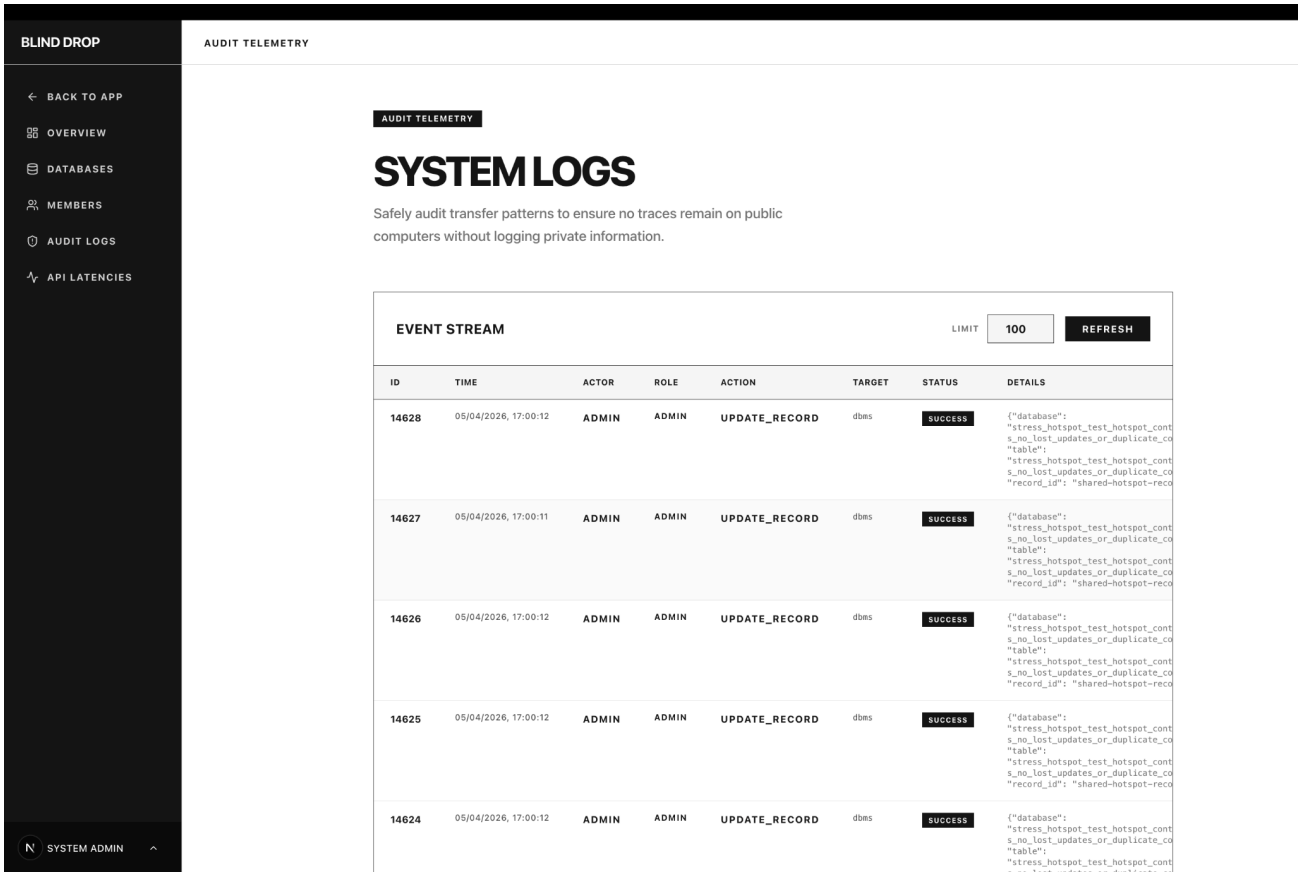

2. After running additional test commands (**run_module_b_tests.py** and **run_module_b_tests.py --run-stress-orchestrator)**, the number of audit log entries increased significantly. The audit logs grew from **3996 entries to 9772 entries**, indicating that further automated and stress testing operations were successfully executed and recorded by the system.

   This increase in audit logs confirms that the testing scripts performed multiple backend and frontend interactions, and all actions were properly tracked within the audit logging mechanism.




3. After running the additional test script (**run_stress_orchestrator.py)**, the number of audit log entries increased further. The audit logs grew from **9,772 entries to 14,628 entries**, indicating that the stress testing operations generated a large number of system interactions which were successfully recorded in the audit logging system.

   This increase confirms that the stress orchestration tests executed multiple backend operations and that all activities were properly tracked by the system's auditing mechanism.




## ACID Verification:-

### Atomicity

- Critical token consumption runs inside explicit transaction scope.
- On any injected or runtime failure, rollback is executed.
- Tests verify no partial token/session/download state is persisted.

### Consistency

- Constraints enforce valid data and relationships:
- Uniqueness on critical keys.
- Foreign keys to prevent orphan references.
- Check constraints for valid statuses and domain rules.
- Concurrent and stress tests verify final counts and key uniqueness invariants.

- Cross-table contention checks confirm no orphaned rows across related tables.

**Isolation**

- Concurrency tests execute many overlapping operations on shared keys.
- Single-winner behavior prevents race corruption.
- RBAC tests ensure unauthorized users cannot mutate protected data.

**Durability**

- Commit path data remains visible after restart simulation from a fresh process.
- Failed operations remain absent across restart checks.
- Audit artifacts persist for post-event traceability.

## Explanations:-

**How correctness of operations is ensured:-** Correctness is ensured using a layered strategy:

- Transaction-safe critical operations with rollback on failure.
- Declarative constraints and uniqueness enforcement in schema.
- Strict role-based authorization checks before sensitive mutations.
- Deterministic concurrency and stress validation using parallel workloads.
- Post-operation verification against persisted database state, not only API responses.
- Continuous audit logging for all major control paths.

**How failures are handled:-**Failure handling includes:

- Controlled fault injection at multiple transaction stages.
- Immediate rollback upon detected exception.
- Error responses that clearly indicate rollback outcome.
- Retriable behavior that succeeds cleanly after rollback.
- Validation that no residual partial writes remain.

**How multi-user conflicts are handled:-**Conflict handling strategy:

- Single-winner update semantics for shared one-time token consumption.

- Conflict status responses for late contenders and duplicate writes.
- Permission isolation between admin and regular users under parallel requests.
- Invariant checks ensure record uniqueness and valid final states.

**Observations:-**

- The system remained stable across all planned contention and load scenarios in recorded runs.
- Race-heavy critical operations consistently produced single-winner outcomes.
- Failure injection proved rollback correctness at multiple transaction points.
- Thousands-scale insert performance was strong, with full success and no invariant violations.
- Audit logging improved confidence by making both normal and abnormal paths observable.

**Limitations:-**

- Tests are executed in a controlled local environment; distributed or networked deployment behavior is not fully characterized.
- SQLite locking and local process assumptions may differ from behavior under clustered deployment.
- Workloads are synthetic and may not cover every real-world traffic pattern.
- Long-duration soak tests beyond the current scope can further strengthen reliability evidence.

**Conclusion:-** Module B objectives were achieved for concurrency safety, race-condition handling, rollback correctness, stress reliability, and ACID-aligned behavior. Across 24 recorded automated tests, all passed with zero failures and zero errors. The system showed robust behavior under parallel access and failure conditions, preserved data correctness, prevented interference between users, and maintained a durable committed state. These outcomes demonstrate that the implementation is reliable and suitable for multi-user operation without breaking correctness guarantees.

## Team Members and Contributions :-

| Members Name | Contribution |
|---|---|
| Thipparapu Rushitha | Module A |
| Sailaja Mudu | Module A |
| Vadithya Harsha Vardhan Nayak | Module A |
| Devvrat Hans | Co-developed Module B features and implementation; contributed to backend/frontend integration and solution ideation. |
| Nitin Bansal | Co-developed Module B features and implementation; handled schema/database integration alignment and overall Module B execution. |